

Code:

```
[org 0x0100]
        jmp start

data:    dw 60, 55, 45, 50, 40, 35, 25, 30, 10, 0
data2:   dw 328, 329, 898, 8923, 8293, 2345, 10, 877, 355, 98
        dw 888, 533, 2000, 1020, 30, 200, 761, 167, 90, 5
swapflag: db 0

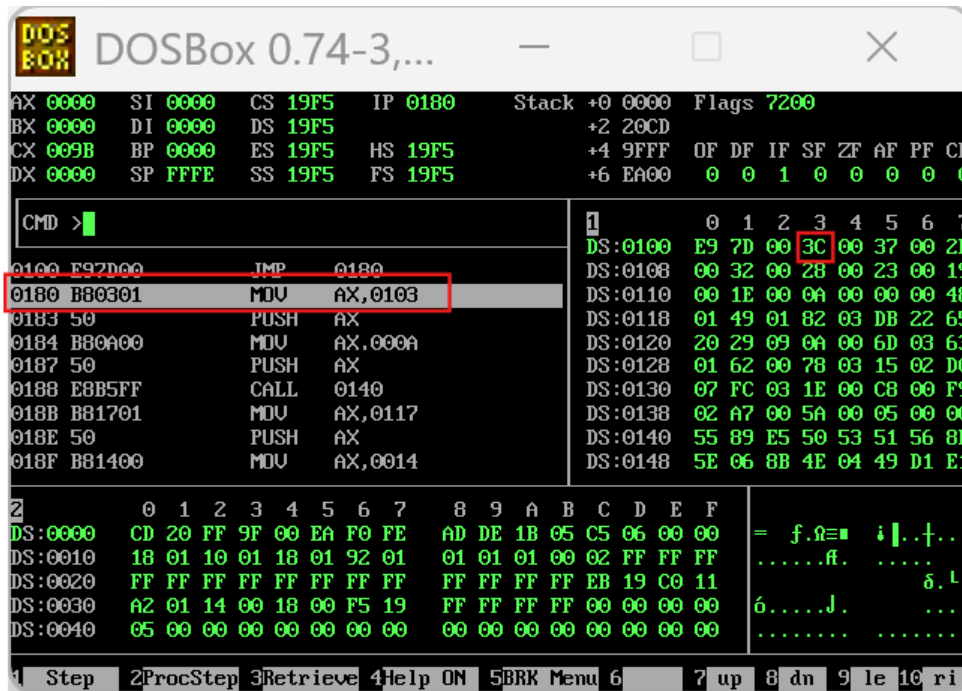
bubblesort:
        push bp
        mov bp, sp
        push ax
        push bx
        push cx
        push si
        mov bx, [bp+6]
        mov cx, [bp+4]
        dec cx
        shl cx, 1
mainloop:
        mov si, 0
        mov byte [swapflag], 0

innerloop:
        mov ax, [bx+si]
        cmp ax, [bx+si+2]
        jbe noswap
        xchg ax, [bx+si+2]
        mov [bx+si], ax
        mov byte [swapflag], 1

noswap:
        add si, 2
        cmp si, cx
        jne innerloop
        cmp byte [swapflag], 1
        je mainloop
        pop si
        pop cx
        pop bx
        pop ax
        pop bp
        ret 4

start:
        mov ax, data
        push ax
        mov ax, 10
        push ax
        call bubblesort
        mov ax, data2
        push ax
        mov ax, 20
        push ax
        call bubblesort
        mov ax, 0x4c00
        int 0x21
```

- 1) The address of data in first label is mov into Ax register (3C=60) in code we could see it is data of our first label.



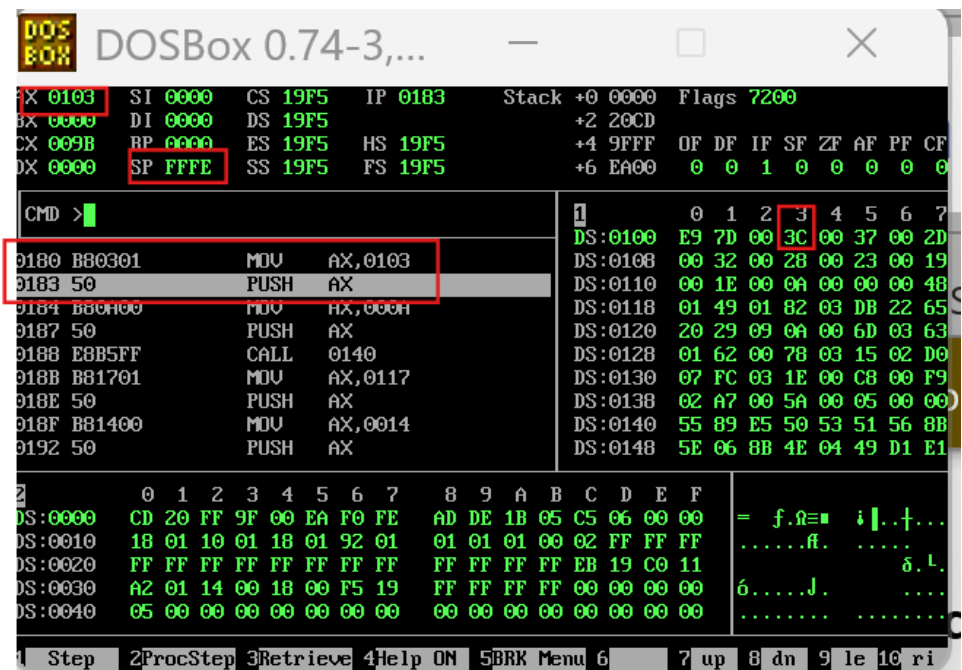
```
DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 0180 Stack +0 0000 Flags 7200
BX 0000 DI 0000 DS 19F5 +2 20CD
CX 009B BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 0000 SP FFFE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 0 0 0 0

CMD >
0100 E92D00 JMP 0180
0180 B80301 MOV AX,0103
0183 50 PUSH AX
0184 B80A00 MOV AX,000A
0187 50 PUSH AX
0188 EB5FFF CALL 0140
018B B81701 MOV AX,0117
018E 50 PUSH AX
018F B81400 MOV AX,0014

DS:0100 E9 7D 00 3C 00 37 00 2D
DS:0108 00 32 00 28 00 23 00 19
DS:0110 00 1E 00 0A 00 00 00 48
DS:0118 01 49 01 82 03 DB 22 65
DS:0120 20 29 09 0A 00 6D 03 63
DS:0128 01 62 00 78 03 15 02 D0
DS:0130 07 FC 03 1E 00 C8 00 F9
DS:0138 02 A7 00 5A 00 05 00 00
DS:0140 55 89 E5 50 53 51 56 8B
DS:0148 5E 06 8B 4E 04 49 D1 E1

2 0 1 2 3 4 5 6 7 8 9 A B C D E F
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.Ω= i |.+.
DS:0010 18 01 10 01 18 01 92 01 01 01 01 00 02 FF FF FF .....ff. ....
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF EB 19 C0 11 .....δ.L
DS:0030 A2 01 14 00 18 00 F5 19 FF FF FF FF 00 00 00 00 6.....J. ....
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..... .....
```

- 2) SP point to our first parameter address now (FFFF-2=FFFE)



```
DOSBox 0.74-3,...
AX 0103 SI 0000 CS 19F5 IP 0183 Stack +0 0000 Flags 7200
BX 0000 DI 0000 DS 19F5 +2 20CD
CX 009B BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 0000 SP FFFE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 0 0 0 0

CMD >
0180 B80301 MOV AX,0103
0183 50 PUSH AX
0184 B80A00 MOV AX,000A
0187 50 PUSH AX
0188 EB5FFF CALL 0140
018B B81701 MOV AX,0117
018E 50 PUSH AX
018F B81400 MOV AX,0014
0192 50 PUSH AX

DS:0100 E9 7D 00 3C 00 37 00 2D
DS:0108 00 32 00 28 00 23 00 19
DS:0110 00 1E 00 0A 00 00 00 48
DS:0118 01 49 01 82 03 DB 22 65
DS:0120 20 29 09 0A 00 6D 03 63
DS:0128 01 62 00 78 03 15 02 D0
DS:0130 07 FC 03 1E 00 C8 00 F9
DS:0138 02 A7 00 5A 00 05 00 00
DS:0140 55 89 E5 50 53 51 56 8B
DS:0148 5E 06 8B 4E 04 49 D1 E1

2 0 1 2 3 4 5 6 7 8 9 A B C D E F
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.Ω= i |.+.
DS:0010 18 01 10 01 18 01 92 01 01 01 01 00 02 FF FF FF .....ff. ....
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF EB 19 C0 11 .....δ.L
DS:0030 A2 01 14 00 18 00 F5 19 FF FF FF FF 00 00 00 00 6.....J. ....
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..... .....
```

3) $SP=SP-2$ ($FFFF-2=FFFD$)

Now AX stores number of elements which is pushed onto stack:

DOSBox 0.74-3,...

Register	Value	Segment	Value	Register	Value	Segment	Value
AX	0103	SI	0000	CS	19F5	IP	0184
BX	0000	DI	0000	DS	19F5		
CX	009B	BP	0000	ES	19F5	HS	19F5
DX	0000	SP	FFFC	SS	19F5	FS	19F5

CMD >

Address	Disassembly	Comment
0183 50	PUSH AX	
0184 B80A00	MOV AX,000A	
0187 50	PUSH AX	
0188 E8B5FF	CALL 0140	
018B B81701	MOV AX,0117	
018E 50	PUSH AX	
018F B81400	MOV AX,0014	
0192 50	PUSH AX	
0193 E8AAFF	CALL 0140	

Address	Hex	ASCII
DS:0100	E9 7D 00 3C 00 37 00 2D	= f.Ω= i .+....
DS:0108	00 32 00 28 00 23 00 19	ff.
DS:0110	00 1E 00 0A 00 00 00 48	δ.L.
DS:0118	01 49 01 82 03 DB 22 65	6.....J.
DS:0120	20 29 09 0A 00 6D 03 63	
DS:0128	01 62 00 78 03 15 02 D0	
DS:0130	07 FC 03 1E 00 C8 00 F9	
DS:0138	02 A7 00 5A 00 05 00 00	
DS:0140	55 89 E5 50 53 51 56 8B	
DS:0148	5E 06 8B 4E 04 49 D1 E1	

1 Step 2ProcStep 3Retrieve 4Help ON 5BRK Menu 6 7 up 8 dn 9 le 10 ri

4) Bubblesort subroutine is called .

The call will automatically push the return address on stack at FFFA seen above at SP

DOSBox 0.74-3,...

Register	Value	Segment	Value	Register	Value	Segment	Value
AX	000A	SI	0000	CS	19F5	IP	0188
BX	0000	DI	0000	DS	19F5		
CX	009B	BP	0000	ES	19F5	HS	19F5
DX	0000	SP	FFFA	SS	19F5	FS	19F5

CMD >

Address	Disassembly	Comment
0187 50	PUSH AX	
0188 E8B5FF	CALL 0140	
018B B81701	MOV AX,0117	
018E 50	PUSH AX	
018F B81400	MOV AX,0014	
0192 50	PUSH AX	
0193 E8AAFF	CALL 0140	
0196 B8004C	MOV AX,4C00	
0199 CD21	INT 21	

Address	Hex	ASCII
DS:0100	E9 7D 00 3C 00 37 00 2D	= f.Ω= i .+....
DS:0108	00 32 00 28 00 23 00 19	ff.
DS:0110	00 1E 00 0A 00 00 00 48	δ.L.
DS:0118	01 49 01 82 03 DB 22 65	6.....J.
DS:0120	20 29 09 0A 00 6D 03 63	
DS:0128	01 62 00 78 03 15 02 D0	
DS:0130	07 FC 03 1E 00 C8 00 F9	
DS:0138	02 A7 00 5A 00 05 00 00	
DS:0140	55 89 E5 50 53 51 56 8B	
DS:0148	5E 06 8B 4E 04 49 D1 E1	

1 Step 2ProcStep 3Retrieve 4Help ON 5BRK Menu 6 7 up 8 dn 9 le 10 ri

- 5) Base pointer is saved by pushing it in stack which will point at subroutine base location
 SP is now brought to BP (array traversed through SP and base address saved through BP so data wont be lost)

DOSBox 0.74-3,...

AX	000A	SI	0000	CS	19F5	IP	0140	Stack	+0 018B	Flags	7200
BX	0000	DI	0000	DS	19F5				+2 000A		
CX	009B	BP	0000	ES	19F5	HS	19F5		+4 0103	OF	DF
DX	0000	SP	FFFB	SS	19F5	FS	19F5		+6 0000	IF	SF
										ZF	AF
										PF	CF

CMD > |

0188	E8B5FF	CALL	0140	DS:0100	E9 7D 00 3C 00 37 00 2D
0140	55	PUSH	BP	DS:0108	00 32 00 28 00 23 00 19
0141	89E5	MOV	BP,SP	DS:0110	00 1E 00 0A 00 00 00 48
0143	50	PUSH	AX	DS:0118	01 49 01 82 03 DB 22 65
0144	53	PUSH	BX	DS:0120	20 29 09 0A 00 6D 03 63
0145	51	PUSH	CX	DS:0128	01 62 00 78 03 15 02 D0
0146	56	PUSH	SI	DS:0130	07 FC 03 1E 00 C8 00 F9
0147	8B5E06	MOV	BX,[BP+06]	DS:0138	02 A7 00 5A 00 05 00 00
014A	8B4E04	MOV	CX,[BP+04]	DS:0140	55 89 E5 50 53 51 56 8B
				DS:0148	5E 06 8B 4E 04 49 D1 E1

2	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
DS:0000	CD	20	FF	9F	00	EA	F0	FE	AD	DE	1B	05	C5	06	00	00
DS:0010	18	01	10	01	18	01	92	01	01	01	01	00	02	FF	FF	FF
DS:0020	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	EB	19	C0	11
DS:0030	A2	01	14	00	18	00	F5	19	FF	FF	FF	FF	00	00	00	00
DS:0040	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

1 Step 2ProcStep 3Retrieve 4Help ON 5BRK Menu 6 7 up 8 dn 9 le 10 ri

- 6) Old value of registers is stored so that we can reuse these registers In our subroutine

DOSBox 0.74-3,...

AX	000A	SI	0000	CS	19F5	IP	0141	Stack	+0 0000	Flags	7200
BX	0000	DI	0000	DS	19F5				+2 018B		
CX	009B	BP	0000	ES	19F5	HS	19F5		+4 000A	OF	DF
DX	0000	SP	FFF6	SS	19F5	FS	19F5		+6 0103	IF	SF
										ZF	AF
										PF	CF

CMD > |

0140	55	PUSH	BP	DS:0100	E9 7D 00 3C 00 37 00 2D
0141	89F5	MOV	BP,SP	DS:0108	00 32 00 28 00 23 00 19
0143	50	PUSH	AX	DS:0110	00 1E 00 0A 00 00 00 48
0144	53	PUSH	BX	DS:0118	01 49 01 82 03 DB 22 65
0145	51	PUSH	CX	DS:0120	20 29 09 0A 00 6D 03 63
0146	56	PUSH	SI	DS:0128	01 62 00 78 03 15 02 D0
0147	8B5E06	MOV	BX,[BP+06]	DS:0130	07 FC 03 1E 00 C8 00 F9
014A	8B4E04	MOV	CX,[BP+04]	DS:0138	02 A7 00 5A 00 05 00 00
014D	49	DEC	CX	DS:0140	55 89 E5 50 53 51 56 8B
				DS:0148	5E 06 8B 4E 04 49 D1 E1

2	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
DS:0000	CD	20	FF	9F	00	EA	F0	FE	AD	DE	1B	05	C5	06	00	00
DS:0010	18	01	10	01	18	01	92	01	01	01	01	00	02	FF	FF	FF
DS:0020	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	EB	19	C0	11
DS:0030	A2	01	14	00	18	00	F5	19	FF	FF	FF	FF	00	00	00	00
DS:0040	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

1 Step 2ProcStep 3Retrieve 4Help ON 5BRK Menu 6 7 up 8 dn 9 le 10 ri

7) $BP+4$ = number of elements, $BP+6$ = address of array now bx and cx store these values

DOSBox 0.74-3,...

AX 000A	SI 0000	CS 19F5	IP 0147	Stack +0 0000	Flags 7200
BX 0000	DI 0000	DS 19F5		+2 009B	
CX 009B	BP FFF6	ES 19F5	HS 19F5	+4 0000	OF DF IF SF ZF AF PF CF
DX 0000	SP FFEE	SS 19F5	FS 19F5	+6 000A	0 0 1 0 0 0 0 0

CMD >

```

0146 56          PUSH    SI
0147 8B5E06       MOV     BX, [BP+06]
014A 8B4E04       MOV     CX, [BP+04]
014D 49          DEC     CX
014E D1E1        SHL     CX, 1
0150 BE0000      MOV     SI, 0000
0153 C6063F0100  MOV     [013F], 00
0158 8B00        MOV     AX, [BX+SI]
015A 3B4002      CMP     AX, [BX+SI+02]
015D 760A        JNA     0169
015F 874002      XCHG    AX, [BX+SI+02]
0162 8900        MOV     [BX+SI], AX
  
```

DS:0100	E9 7D 00 3C 00 37 00 2D	
DS:0108	00 32 00 28 00 23 00 19	
DS:0110	00 1E 00 0A 00 00 00 48	
DS:0118	01 49 01 82 03 DB 22 65	
DS:0120	20 29 09 0A 00 6D 03 63	
DS:0128	01 62 00 78 03 15 02 D0	
DS:0130	07 FC 03 1E 00 C8 00 F9	
DS:0138	02 A7 00 5A 00 05 00 00	
DS:0140	55 89 E5 50 53 51 56 8B	
DS:0148	5E 06 8B 4E 04 49 D1 E1	

Step 2ProcStep 3Retrieve 4Help ON 5BRK Menu 6 7 up 8 dn 9 le 10 ri

8) Now sorting occurs (already explained in previous labs):

DOSBox 0.74-3,...

AX 000A	SI 0000	CS 19F5	IP 014E	Stack +0 0000	Flags 7204
BX 0103	DI 0000	DS 19F5		+2 009B	
CX 0009	BP FFF6	ES 19F5	HS 19F5	+4 0000	OF DF IF SF ZF AF PF CF
DX 0000	SP FFEE	SS 19F5	FS 19F5	+6 000A	0 0 1 0 0 0 1 0

CMD >

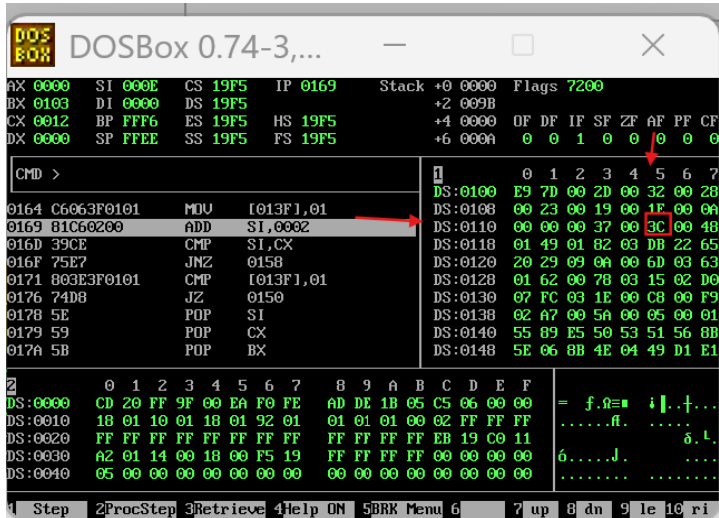
```

014D 49          DEC     CX
014E D1E1        SHL     CX, 1
0150 BE0000      MOV     SI, 0000
0153 C6063F0100  MOV     [013F], 00
0158 8B00        MOV     AX, [BX+SI]
015A 3B4002      CMP     AX, [BX+SI+02]
015D 760A        JNA     0169
015F 874002      XCHG    AX, [BX+SI+02]
0162 8900        MOV     [BX+SI], AX
  
```

DS:0100	E9 7D 00 3C 00 37 00 2D	
DS:0108	00 32 00 28 00 23 00 19	
DS:0110	00 1E 00 0A 00 00 00 48	
DS:0118	01 49 01 82 03 DB 22 65	
DS:0120	20 29 09 0A 00 6D 03 63	
DS:0128	01 62 00 78 03 15 02 D0	
DS:0130	07 FC 03 1E 00 C8 00 F9	
DS:0138	02 A7 00 5A 00 05 00 00	
DS:0140	55 89 E5 50 53 51 56 8B	
DS:0148	5E 06 8B 4E 04 49 D1 E1	

Step 2ProcStep 3Retrieve 4Help ON 5BRK Menu 6 7 up 8 dn 9 le 10 ri

- 9) After some iterations we see that 3C has moved it location and stopped moving further showing the data has been sorted.



```
DOSBox 0.74-3,...
```

AX	SI	DI	CS	IP	Stack	Flags
0000	000E	0000	19F5	0169	+0 0000	7200
0103	0000	0000	19F5		+2 009B	
0012	FFFF	0000	19F5		+4 0000	OF DF IF SF ZF AF PF CF
0000	FFEE	0000	19F5		+6 000A	0 0 1 0 0 0 0

```
CMD >
```

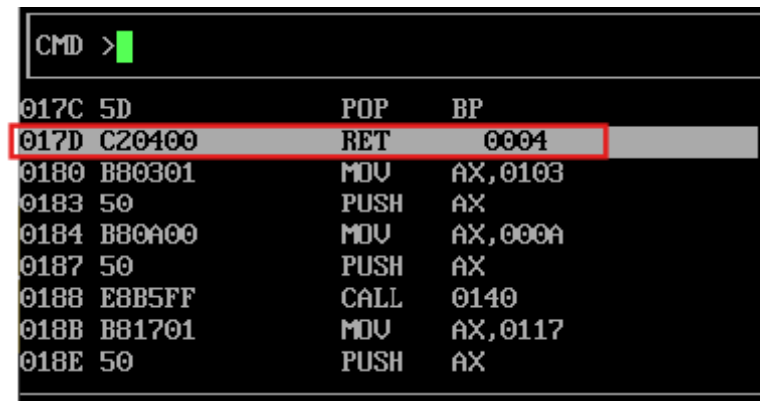
0164	C6063F0101	MOV	[013F1,01
0169	01C60200	ADD	SI,0002
016D	39CE	CMP	SI,CX
016F	75E7	JNZ	0158
0171	003E3F0101	CMP	[013F1,01
0176	74D8	JZ	0150
0178	5E	POP	SI
0179	59	POP	CX
017A	5B	POP	EX

DS:0100	E9 7D 00 2D 00 32 00 28
DS:0108	00 23 00 19 00 1F 00 0A
DS:0110	00 00 00 37 00 3C 00 48
DS:0118	01 49 01 82 03 DB 22 65
DS:0120	20 29 09 0A 00 6D 03 63
DS:0128	01 62 00 78 03 15 02 D0
DS:0130	07 FC 03 1E 00 C8 00 F9
DS:0138	02 A7 00 5A 00 05 00 01
DS:0140	55 89 E5 50 53 51 56 8B
DS:0148	5E 06 8B 4E 04 49 D1 E1

DS:0000	CD 20 FF 9F 00 EA F0 FE	AD DE 1B 05 C5 06 00 00	= f.n=	↓	↑	...
DS:0010	18 01 10 01 18 01 92 01	01 01 01 00 02 FF FF FF	R.			
DS:0020	FF FF FF FF FF FF FF	FF FF FF FF EB 19 C0 11				δ.L.
DS:0030	A2 01 14 00 18 00 F5 19	FF FF FF FF 00 00 00 00	6.....J.			
DS:0040	05 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00				

```
1 Step 2ProcStep 3Retrieve 4Help ON 5BRK Menu 6 7 up 8 dn 9 le 10 ri
```

- 10) Finally our final data will be returned and program will terminate.



```
CMD >
```

017C	5D	POP	BP
017D	C20400	RET	0004
0180	B80301	MOV	AX,0103
0183	50	PUSH	AX
0184	B80A00	MOV	AX,000A
0187	50	PUSH	AX
0188	E8B5FF	CALL	0140
018B	B81701	MOV	AX,0117
018E	50	PUSH	AX

Task 2: Differences from 5.6 (theory)

- **Extra 2 bytes of empty space on the stack for swapflag.**
- **inside the loop it accesses that 2 bytes of reserved space.**
- **Instead of using global memory now it uses local stack variable (in example 5.6).**
- **In simple words, the major difference is addressing scheme**